Trevor Labat
Student ID: 801340975
GitHub: https://github.com/TlabatCharlotte/Machine-Learning

Homework #4

Problem 1 - Support Vector Machine Classifier for Cancer Dataset

Using train_test_split method from sklearn.model_selection the cancer data is split into a training set 80% and 20% validation set. Standardization is immediately applied afterwards using the sklearn.preprocessing library.

Assumptions made: The StandardScaler is fit on the training data only and then applied to the validation data. This is to prevent information leakage as fitting the validation data would not fairly test the model for new data. C, gamma, and degree values were left to the defaults assigned by the SVM importation as the homework assignment never asked to choose C, gamma, or degree. Kernels compared were sigmoid, linear, polynomial, and the radial basis function (RBF). Comparison with HW 3 uses the logistic regression results with accuracy = 0.991, precision = 0.971, and recall = 1.0 using a value of 10 for lambda. Label mapping: M = 1, B = 0. Random_state = 4.

The sklearn library is used to import the SVM module for accessing the support vector classifier along with the metric module used for calculating the accuracy, precision, and recall scores for all 4 kernels. The code highlighted throughout the report will be for the linear kernel, but it is identical for each kernel. The implementation for the linear kernel can be seen in Figure 1.1.

```
from sklearn import svm

linear_classifier = svm.SVC(kernel='linear')
linear_classifier.fit(X_train, Y_train)
Y_pred_linear = linear_classifier.predict(X_test)
```

Figure 1.1 - Linear classifier using sklearn library

```
from sklearn import metrics

linear_acc  = metrics.accuracy_score(Y_test, Y_pred_linear)
linear_prec = metrics.precision_score(Y_test, Y_pred_linear)
linear_rec  = metrics.recall_score(Y_test, Y_pred_linear)
print("Linear accuracy", linear_acc, "Linear precision", linear_prec,
      "Linear recall", linear_rec)
```

Figure 1.2 - Linear classifier metrics using sklearn

The same is done for the polynomial, sigmoid, and RBF kernels. And all four results are individually plotted in a bar chart and then plotted all together in a single bar chart along with the logistic regression results from HW 3. Classification metrics for each kernel can be seen in

Figures 1.3, 1.4, 1.5, and 1.6 respectively along with the final comparison with HW 3 in Figure 1.7.

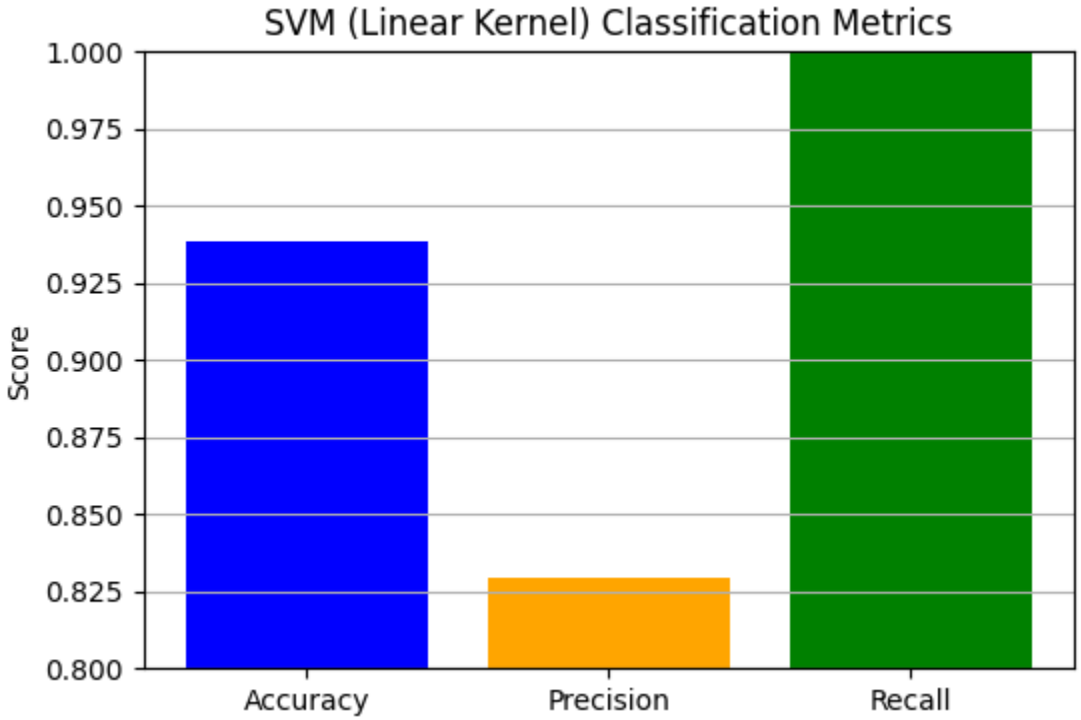


Figure 1.3 - Linear kernel classification metrics

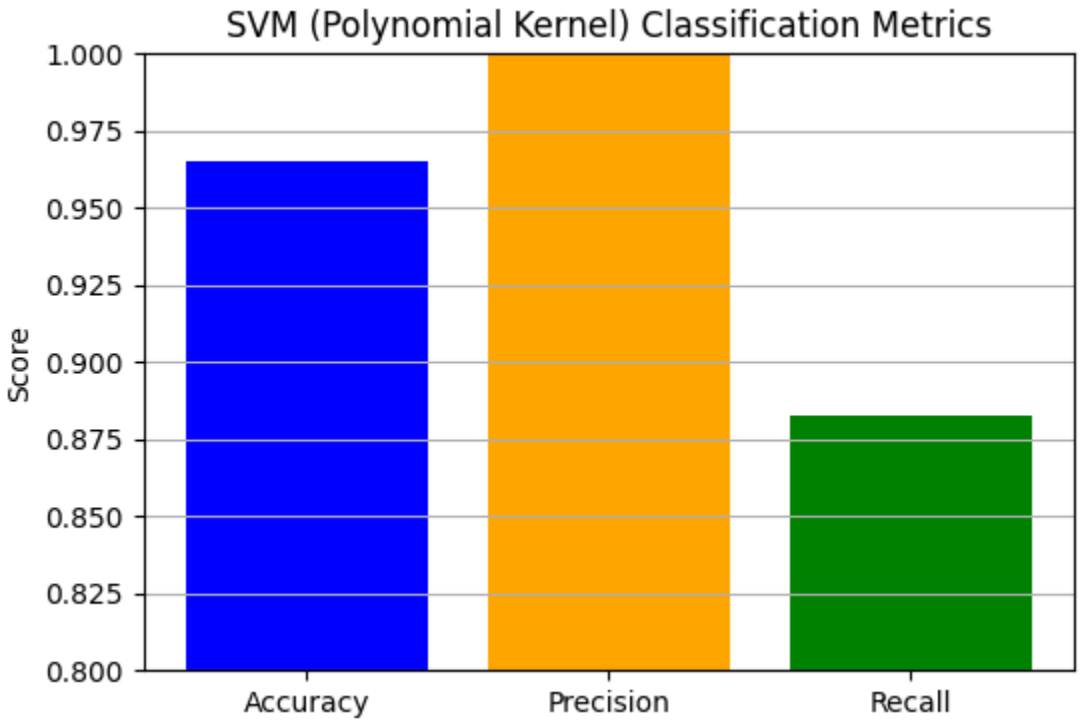


Figure 1.4 - Polynomial kernel classification metrics

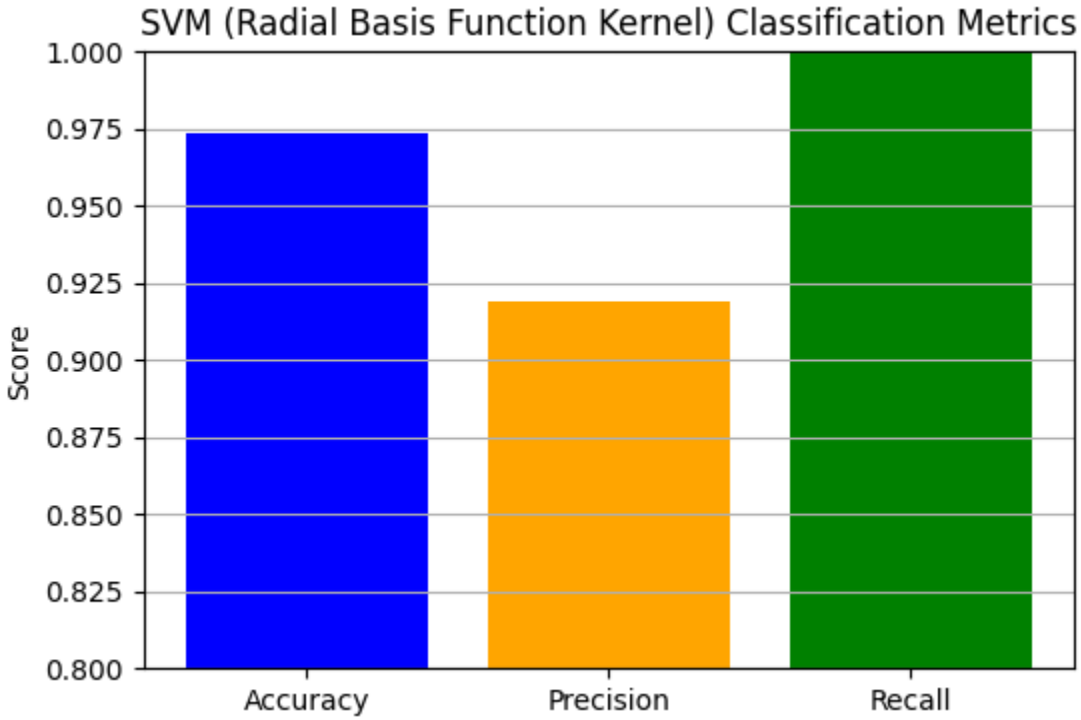


Figure 1.5 - RBF kernel classification metrics

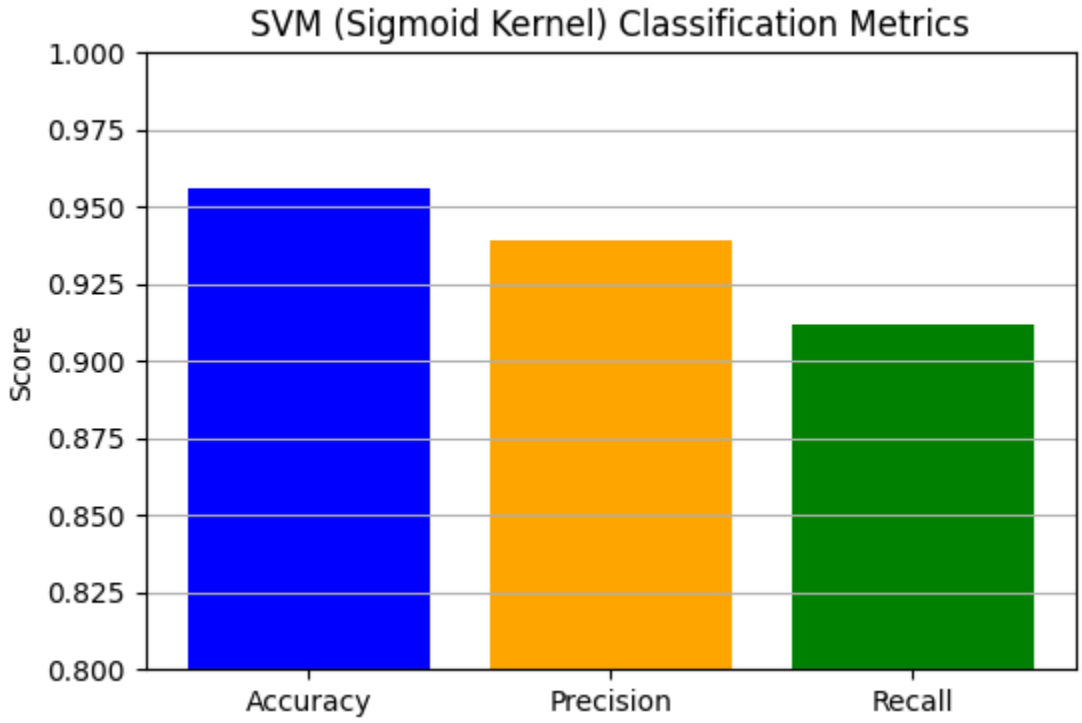


Figure 1.6 - Sigmoid kernel classification metrics

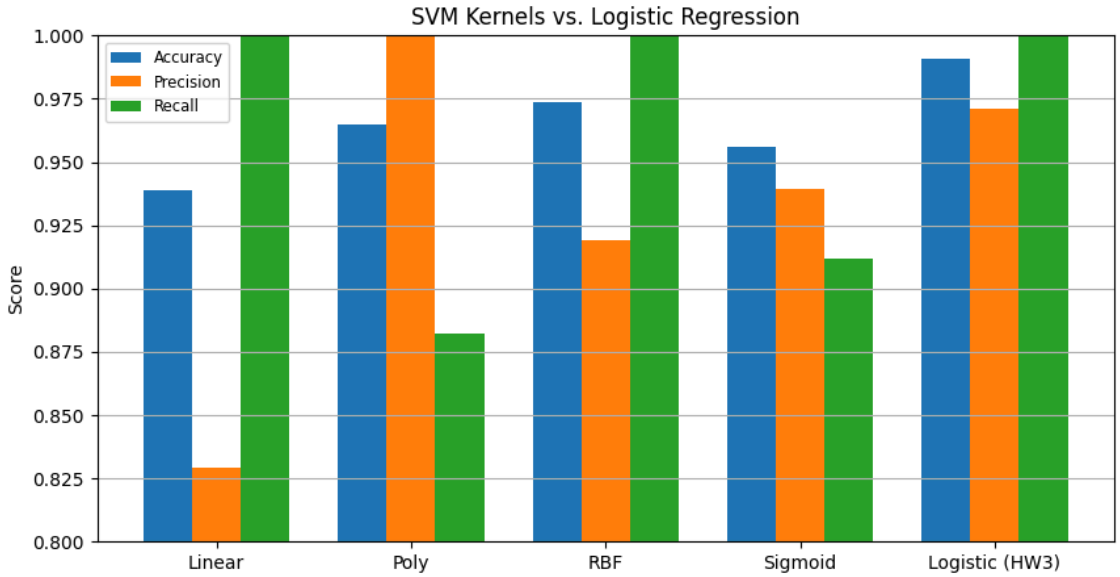


Figure 1.7 - All kernel classification validation metrics compared with HW 3 validation metrics.

The final scores for each kernel classification are the following (rounded to 5 decimal places):

- Linear kernel metrics
    - Accuracy: 0.93860
    - Precision: 0.82927
    - Recall: 1.0
- Polynomial kernel metrics
    - Accuracy: 0.96491
    - Precision: 1.0
    - Recall: 0.88235
- RBF kernel metrics
    - Accuracy 0.97368
    - Precision: 0.91891
    - Recall 1.0
- Sigmoid kernel metrics
    - Accuracy: 0.95614
    - Precision 0.93939
    - Recall 0.91176
- HW 3 cancer results (lambda = 10)
    - Accuracy: 0.99123
    - Precision: 0.97143
    - Recall: 1.0

Because the penalty parameter was not tuned, unlike lambda in HW 3, the comparison between the SVM and logistic regression models is not totally fair. If the value of C were to be tuned for the SVM then it is possible one of the kernels would outperform the logistic regression used in HW 3. But this wasn't tested.

With the default parameters, the RBF was the best performing support vector machine kernel with an accuracy of 0.97368 and a recall of 1.0. The logistic regression model has the highest accuracy, the polynomial kernel has the highest precision, and logistic regression is second on precision.

When considering the metrics, recall matters the most because missing a malignant tumor is worse than a false alarm. RBF and linear both reached a recall of 1.0, but RBF had higher accuracy and precision.

Problem 2 - Support Vector Regression for Housing Dataset

Using train_test_split method from sklearn.model_selection the housing data is split into a training set 80% and 20% validation set. Standardization is immediately applied afterwards using the sklearn.preprocessing library.

Assumptions made: The StandardScaler is fit on the training data only and then applied to the validation data. This is to prevent information leakage as fitting the validation data would not fairly test the model for new data. Random_state = 100 (same as HW 2 for the same data split). All 11 features were used as explanatory variables. The yes/no values were mapped to ones and zeros. Predictions were converted back to dollars. The kernel parameters were taken from the sample code and were not tuned: C = 1e3, gamma = 0.1, and degree = 2. The parameters were decided by the library for the sigmoid kernel that was mentioned in the slides.

The assignment says HW 1, but the housing data was used in HW 2, so HW 2 was used. HW 2 ran 2000 iterations with a lambda of 0.001.

The sklearn library is used for all 4 SVR kernels and for the metric module to calculate the mean squared error and R2 score used on the validation set for comparing the kernels with HW 2's regularized linear regression model. The code for each kernel can be seen in Figure 2.1.

```
from sklearn.svm import SVR

svr_rbf = SVR(kernel='rbf', C=1e3, gamma=0.1)
svr_lin = SVR(kernel='linear', C=1e3)
svr_poly = SVR(kernel='poly', C=1e3, degree=2)
svr_sigmoid = SVR(kernel='sigmoid')

y_rbf = svr_rbf.fit(X_train, Y_train_s).predict(X_test)
y_lin = svr_lin.fit(X_train, Y_train_s).predict(X_test)
y_poly = svr_poly.fit(X_train, Y_train_s).predict(X_test)
y_sigmoid = svr_sigmoid.fit(X_train, Y_train_s).predict(X_test)
```

Figure 2.1 - SVR using RBF, linear, poly, and sigmoid kernels

The code for obtaining the R2 and MSE metrics can be found in Figure 2.2. Note that the root of the resulting MSE values was taken in order to get the error in dollars. The values below show the calculated RMSE and the R2 values.

- Linear

    - RMSE: 1070650
    - R2: 0.6752247600322347
- Poly
    - RMSE: 1355390
    - R2: 0.47950009143796235
- RBF
    - RMSE: 2299440
    - R2 : -0.4980778837440929
- Sigmoid
    - RMSE: 3936350
    - R2: -3.390131333260114
- HW 2
    - RMSE: 1117970
    - R2: 0.6458860294045985

```
from sklearn import metrics
models = [('Linear', y_lin), ('Poly', y_poly), ('RBF', y_rbf), ('Sigmoid', y_sigmoid)]
for name, p in models:
    print(name, "MSE", metrics.mean_squared_error(Y_test, p), "R2", metrics.r2_score(Y_test, p))
```

Figure 2.2 - R2 and MSE metrics for each kernel

Looking at these results we see that the linear kernel is best in the SVR. The RMSE is the lowest at $1.07M and the R2 score is the highest at 0.675 (1 = perfect and 0 = basically guessing the average price) and beats HW 2 slightly (this may be because HW 2 may not have fully converged). Poly is worse than linear, RBF is worse than baseline (worse than just guessing the average), and sigmoid is far worse. This may be attributed to the untuned parameters like C, gamma, and degree but this wasn't done as it was out of scope for the assignment. The comparison is for “untuned parameters” not for the best possible kernel. The R2 scores can be easily viewed in Figure 2.3. Note that the sigmoid kernel is literally off the charts (goes beyond the y limit of -1).

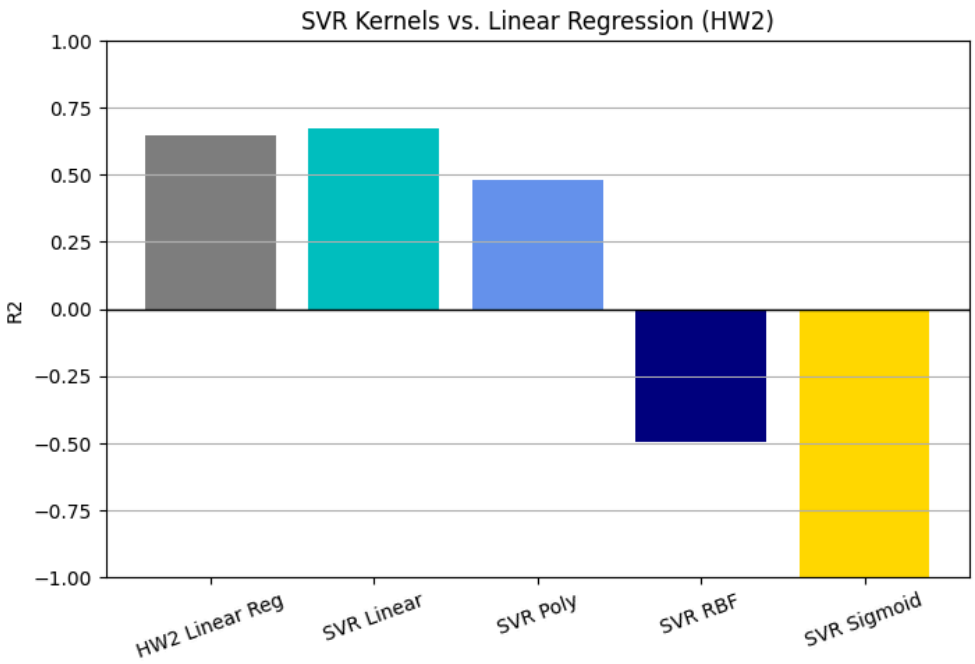


Figure 2.3 - R2 score for all 4 kernels and HW2

In order to visualize the accuracy of the kernels, a plot of area vs price was created to match the SVR scatter plot in the SVR sample code that plots one feature against the target. A set of evenly spaced area values is created and used to make a 200x11 feature matrix of zeros with column zero being replaced with the area. Each SVR is run on this matrix and converted into dollars with the price scaler. The area is also converted back to real units and the actual validation houses are plotted as orange dots and each kernel's prediction as a line. See Figure 2.4 for the code implementation and Figure 2.5 for the resulting plot.

Note the Price vs. Area plot only considers one of the 11 features, so a good looking look doesn't contradict the previously mentioned scores. The lines represent a trend and do not fit the individual houses.

```
area_grid = np.linspace(X_test[:, 0].min(), X_test[:, 0].max(), 200)
X_grid = np.zeros((200, 11))
X_grid[:, 0] = area_grid
area_real = scaler.mean_[0] + area_grid * scaler.scale_[0]

def pred(m):
    return Y_scaler.inverse_transform(m.predict(X_grid).reshape(-1, 1)).ravel()

plt.scatter(housing_validation['area'], Y_test, color='darkorange', label='data')
plt.plot(area_real, pred(svr_rbf), color='navy', lw=2, label='RBF model')
plt.plot(area_real, pred(svr_lin), color='c', lw=2, label='Linear model')
plt.plot(area_real, pred(svr_poly), color='cornflowerblue', lw=2, label='Polynomial model')
plt.plot(area_real, pred(svr_sigmoid), color='gold', lw=2, label='Sigmoid model')
plt.xlabel('Area')
plt.ylabel('Price')
plt.title('Support Vector Regression')
plt.legend()
plt.show()
```

Figure 2.4 - Price vs. area

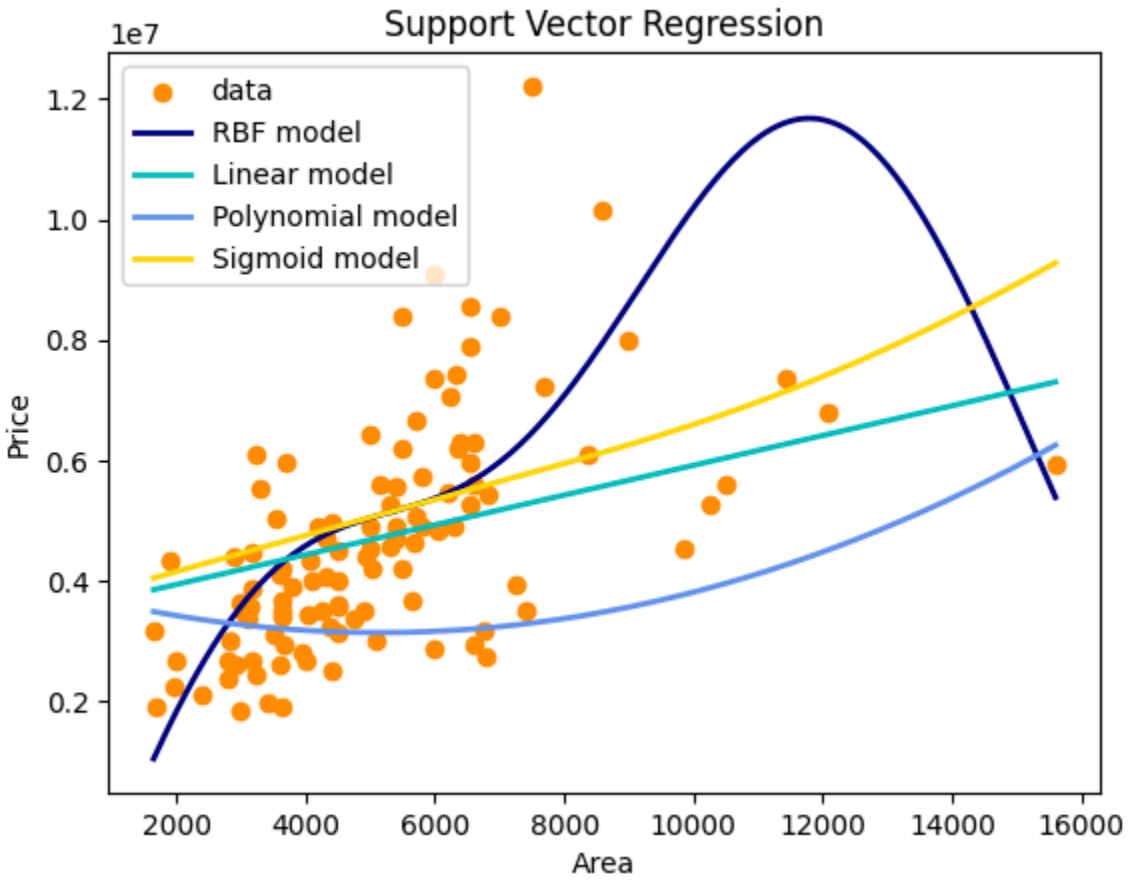


Figure 2.5 - Price vs. area (other features are averaged)